D003

Asset System Architecture

D003 establishes the complete Asset System of Echo Engine. Every engine subsystem depends on assets, but the asset layer itself must remain focused on discovery, import, loading, validation, caching, referencing, unloading, and lifecycle tracking. It is independent of rendering, animation, ECS, physics, and gameplay.

Field	Summary
Objective	Define the canonical asset layer and the contracts that later milestones must obey.
Scope	AssetManager, AssetRegistry, AssetLoader, AssetCache, AssetService, AssetHandle, AssetDescriptor, AssetState, AssetType, loaders, and lifecycle policy.
Non-scope	Rendering, animation runtime, ECS, physics simulation, and gameplay semantics.
Outcome	A modular asset backbone with explicit ownership, dependency tracking, and freeze criteria.

The goal of D003 is to make all resource-related behavior predictable. The system must know what exists, where it came from, how it is validated, how it is referenced, and when it can be safely released. Nothing else should be allowed to blur that responsibility.

1. Asset System Mandate

The asset layer is the canonical source of truth for engine resources.

The asset system is not a convenience layer. It is the engine service that protects every downstream subsystem from ad hoc file access, duplicated loading logic, and unclear ownership. If a subsystem needs a resource, it should request it from the asset system through explicit contracts.

D003 exists to keep resources consistent across the engine. That means the asset system must track the lifecycle of each asset from discovery to unload, and it must do so without assuming how a renderer, audio engine, or gameplay system will consume the asset later.

The architecture must be narrow in purpose but broad in support. It should be responsible for resource identity, metadata, dependency tracking, and lifecycle policy, but not for how a resource becomes visible on screen or audible in the mixer.

Asset system rules:

• own resource lifecycle, identity, caching, and dependency visibility

• remain independent of rendering, animation, ECS, physics, and gameplay policy

• provide explicit handles and descriptors rather than hidden global references

• validate resources before they become available to consumers

• support streaming readiness and future package-oriented delivery

The asset system is the resource backbone of Echo Engine. Later milestones may specialize consumption, but they must not replace asset ownership.

2. Scope and Directory Boundaries

The directory structure expresses the asset architecture and should be preserved.

The source tree under src/assets maps directly onto the architecture. Core asset classes live at the root of the asset directory. Loader specializations live under loaders/. This separation prevents the generic asset model from being polluted by format-specific behavior.

D003 does not redesign earlier milestones. Instead, it defines how asset responsibilities fit into the engine backbone so that framework, runtime, renderer, audio, and gameplay can all depend on the same resource language.

File	Role
AssetManager.js	Top-level orchestration, lifecycle coordination, and service delegation.
AssetRegistry.js	Identity map, descriptor catalog, and dependency index.
AssetLoader.js	Generic loader contract and specialized loader base behavior.
AssetCache.js	Cache policy, duplicate prevention, and reference-friendly storage.
AssetService.js	Framework-facing service wrapper for asset operations.
AssetHandle.js	Consumer-facing reference object with safe asset access semantics.
AssetDescriptor.js	Metadata and import/runtime identity information.
AssetState.js	Lifecycle state model and transition vocabulary.
AssetType.js	Type classification for loaders, validation, and routing.

The loaders folder is a special extension point. It holds asset-format specialists such as TextureLoader, ModelLoader, ShaderLoader, AudioLoader, VideoLoader, FontLoader, and MaterialLoader. Each loader may understand its own format, but all of them must obey the same asset lifecycle contract.

3. Asset Dependency Model

The dependency flow keeps ownership centralized and resource use explicit.

The asset system must use a one-directional dependency chain so that higher-level ownership stays clear. AssetManager orchestrates the whole system. AssetRegistry records what exists. AssetLoader chooses how to import or load it. Specialized loaders do the format work. AssetCache manages reuse. AssetHandle exposes access to consumers.

This direction exists to prevent resource ownership confusion. The registry should not need to know loader internals. A loader should not directly become the consumer-facing API. A cache should not decide how assets are discovered. That separation keeps the design maintainable and testable.

Layer	Owns	Reason
AssetManager	Central orchestrator.	Owns the overall asset lifecycle.
AssetRegistry	Descriptor and dependency catalog.	Knows what assets exist and how they relate.
AssetLoader	Loader contract and routing layer.	Knows how to import and load classes of assets.
Specialized loaders	Format-specific import logic.	Handle texture, model, shader, audio, video, font, and material variants.
AssetCache	Reuse and duplicate prevention.	Stores loaded assets according to policy.
AssetHandle	Consumer-facing reference.	Provides controlled access to asset content.

This flow is intentional. It allows the engine to answer three questions at all times: what does the asset system know, what is currently loaded, and who is allowed to use it.

4. Core Asset Classes

The main classes define the authoritative asset vocabulary.

The core asset classes form the vocabulary used by every subsystem. Their job is not to implement loading tricks. Their job is to make identity, state, and ownership unambiguous across the engine.

Each class should have one clearly documented responsibility. If a class starts to combine identity, loading, and storage policy, the architecture becomes harder to reason about and more difficult to freeze.

AssetManager

AssetManager is the top-level coordinator for the entire asset system.

Aspect	Architecture guidance
Purpose	Own the public asset workflow and coordinate discovery, loading, caching, and unload decisions.
Responsibilities	Manage registry, loader selection, cache policy, validation flow, and handle creation.
Ownership	Owns the system-level asset lifecycle and policy, not the raw asset data itself.
Lifetime	Lives with the asset subsystem and shuts down after dependent services release their handles.
Dependencies	Depends on registry, cache, loader interfaces, state, and diagnostics.
Public API	Register, request, release, unload, inspect status, and query dependency state.
What it owns	Registry references, lifecycle policy, and the active coordination state.
What it never owns	Renderer-specific objects, gameplay rules, or feature-system semantics.

AssetRegistry

AssetRegistry is the catalog of known assets and their relationships.

Aspect	Architecture guidance
Purpose	Track descriptors, identifiers, dependency edges, source identity, and import metadata.
Responsibilities	Index assets, resolve references, detect duplicates, and support validation.
Ownership	Owns catalog data, not loaded binary payloads.
Lifetime	Persists as long as the asset system must resolve references.
Dependencies	Depends on descriptors, states, and loader policy, not on consumers.
Public API	Register descriptor, resolve by ID/path/type, enumerate dependencies, and validate catalog.
What it owns	Descriptors, identifiers, dependency graph data, and source metadata.
What it never owns	Concrete loaded asset memory or render-time state.

AssetLoader

AssetLoader is the generic contract used by all import and load specializations.

Aspect	Architecture guidance
Purpose	Define a consistent high-level loading contract for all asset formats.
Responsibilities	Validate inputs, import source data, create runtime-ready asset content, and report failure modes.
Ownership	Owns loader behavior and format policy, not asset storage.
Lifetime	May be long-lived as a service or short-lived as a strategy object depending on composition policy.
Dependencies	Depends on descriptors, type routing, cache policy, and diagnostics.
Public API	CanLoad, Import, Load, Validate, and optional Streaming readiness signals.
What it owns	Loader-specific assumptions, validation rules, and import behavior.
What it never owns	Cache policy, handle lifetime, or consumer ownership.

AssetCache

AssetCache manages loaded asset reuse and duplicate prevention.

Aspect	Architecture guidance
Purpose	Store resolved assets so repeated requests reuse the same underlying resource when appropriate.
Responsibilities	Cache by identity, enforce deduplication, support preloading, and release unused entries.
Ownership	Owns cached entries and their cache policy metadata.
Lifetime	Lives with the asset system and clears with shutdown or cache invalidation.
Dependencies	Depends on handles, descriptors, asset state, and reference-count policy.
Public API	Get, Put, Contains, Evict, Clear, and usage inspection methods.
What it owns	Cached asset entries and cache metadata.
What it never owns	Loader logic or import-source interpretation.

AssetService

AssetService is the framework-facing service wrapper for asset access.

Aspect	Architecture guidance
Purpose	Expose asset capabilities to the rest of the engine through a service boundary.
Responsibilities	Start and stop the asset system, expose request APIs, and coordinate with framework lifecycle.
Ownership	Owns integration with framework services, not asset format details.
Lifetime	Managed by the framework service lifecycle.
Dependencies	Depends on AssetManager, framework service contracts, and diagnostics.
Public API	Initialize, update, request, release, inspect, and shutdown operations.
What it owns	Service-level integration and entry points.
What it never owns	Concrete import logic or direct memory control of unrelated systems.

AssetHandle

AssetHandle is the consumer-facing reference to a managed asset.

Aspect	Architecture guidance
Purpose	Provide safe access to an asset without leaking ownership or storage internals.
Responsibilities	Reference acquisition, release signaling, validity checks, and identity access.
Ownership	Owns the consumer link to the asset, not the asset itself.
Lifetime	Exists while a consumer holds or observes the asset.
Dependencies	Depends on AssetManager, AssetState, and registry identity.
Public API	IsValid, Get, Release, RefCount view, and identity query methods.
What it owns	A reference contract and state about the consumer relationship.
What it never owns	The asset payload, source file, or importer state.

5. Descriptor, State, and Type Model

Descriptors and state types keep the asset layer explicit and inspectable.

AssetDescriptor, AssetState, and AssetType are the supporting data model that allows the asset system to stay deterministic. They are not cosmetic structures. They are the basis for validation, caching, dependency tracking, and future build pipeline compatibility.

The descriptor model should preserve both source identity and runtime identity. That means the system can explain where an asset came from, how it was imported, what type it is, and which other assets it depends on.

Type	Architecture role
AssetDescriptor	Carries asset identity, source path, import path, runtime path, version, platform variants, and compression notes.
AssetState	Defines the lifecycle vocabulary: discovered, registered, importing, validated, loading, ready, referenced, releasing, unloaded, failed.
AssetType	Classifies assets for loader routing, validation policy, and package behavior.

A good descriptor tells the whole story of the asset lifecycle without involving the loader implementation. A good state type tells the whole story of asset availability without involving the consumer. A good type enum keeps routing predictable without pretending to solve the actual format problem.

6. Loader Architecture

Generic and specialized loaders cooperate through a single policy surface.

The loader architecture is layered. AssetLoader defines the generic contract. Specialized loaders implement asset-specific import and validation behavior. The asset manager selects the appropriate loader by type, extension, metadata, or registration policy.

Specialized loaders are required because texture, model, shader, audio, video, font, and material assets have different validation and packaging needs. The architecture should not force one generic loader to understand all formats equally well.

Loader	Role
TextureLoader	Handles image inputs, color-space constraints, compression readiness, and streaming hints.
ModelLoader	Handles geometry inputs, mesh metadata, skinning readiness, and dependency resolution.
ShaderLoader	Handles shader source, compile-time validation, variant readiness, and platform compatibility.
AudioLoader	Handles sample formats, duration metadata, streaming readiness, and channel policy.
VideoLoader	Handles playback metadata, container compatibility, and streaming-oriented import policy.
FontLoader	Handles typeface metadata, font-face validation, and runtime packaging readiness.
MaterialLoader	Handles material descriptors, referenced textures, shader links, and variant policy.

How new loaders are added: a new format gets a loader class, a type registration rule, a validation contract, and a clear place in the import pipeline. That is enough to extend the system without rewriting the manager or changing the meaning of handles.

Plugin readiness means the loader layer can eventually accept externally supplied loaders if they obey the same contract and registration rules. The architecture should not require plugins today, but it must not block them tomorrow.

7. Asset Lifecycle and Memory Architecture

Discovery through cleanup must be explicit, reference-aware, and leak-resistant.

An asset must pass through a defined lifecycle so the engine can reason about memory ownership. Discovery locates the asset. Registration makes it known. Import converts the source into managed content. Validation ensures the content is safe. Loading makes it available. Caching manages reuse. Reference acquisition tracks consumer demand. Release, unloading, and cleanup reclaim memory when use ends.

The memory model should prefer reference counting for shared assets because many engine subsystems will point at the same resource at once. The cache should prevent duplicate loads, but handles and reference policy should determine when an asset can actually be freed.

Phase	What happens	Why it matters
Discovery	Find the source asset and derive initial metadata.	No payload is loaded yet.
Registration	Record the descriptor and its identifiers.	Asset becomes known to the registry.
Import	Convert source into engine-ready content.	Loader-specific work begins.
Validation	Check format, metadata, dependencies, and compatibility.	Failures stop the lifecycle early.
Loading	Materialize the runtime asset content.	Loaded content can be cached.
Caching	Store reusable loaded content.	Duplicate prevention begins.
Reference acquisition	Issue or update handles and reference counts.	Consumers can use the asset.
Usage	Asset remains valid while references exist.	The system tracks active demand.
Release	Consumers drop handles or decrement references.	Asset becomes eligible for unload.
Unloading	Remove runtime content when no longer needed.	Memory is reclaimed safely.
Cleanup	Clear residual state and finalize metadata.	The lifecycle closes cleanly.

Lazy loading and preloading are both allowed, but they must obey the same reference rules. Streaming readiness is a future-facing contract that keeps the system ready for large worlds, large media, and background loading without tying D003 to a particular streaming engine.

8. Asset Metadata and Dependency Tracking

Metadata makes assets inspectable, portable, and safe to reproduce.

Asset metadata is the evidence trail of the asset system. It records the UUID, source path, import path, runtime path, version, compression information, platform variants, and any dependency edges needed to rebuild or validate the asset later.

Dependency tracking must support both direct and indirect references so the system can avoid circular references, understand package closure, and unload assets in a safe order. The registry owns the graph; the cache and handles consume the graph; loaders contribute dependency information during import.

Metadata field	Meaning
UUID	Stable identity used to refer to an asset independent of file name or location.
Source path	The canonical path or origin used for import and reimport.
Import path	The transformed or staged input used by the importer pipeline.
Runtime path	The packaged or resolved path used by engine consumers.
Version	Tracks format revisions and import compatibility.
Compression info	Describes storage, packing, and streaming-friendly compression state.
Platform variants	Describes differences required by target platforms or build targets.

The metadata model also supports future asset database, build pipeline, and package tooling. If a property is needed for those future systems, it should be expressed here now rather than encoded later as a hidden convention.

9. Error Handling and Recovery Strategy

The asset system must fail visibly, classify the failure, and recover where possible.

Asset failure should be explicit. Invalid assets should be rejected early. Missing assets should be reported with enough metadata to trace the request. Corrupted assets should fail validation. Circular references should be detected by the registry or loader dependency pass. Loader failures should be reported through a structured error path rather than a silent null return.

Recovery strategy depends on the failure type. Some failures are recoverable by retry or fallback. Others should stop the load path immediately. The architecture must distinguish between a transient loading problem and a broken asset package or malformed descriptor.

Failure type	Response
Invalid asset	Descriptor or metadata is structurally wrong and should not be loaded.
Missing asset	Requested asset cannot be located by registry or source path.
Corrupted asset	The asset contents fail validation or cannot be parsed correctly.
Circular reference	Dependency graph loops back and cannot be safely resolved.
Import failure	Loader cannot transform the source into a valid runtime asset.
Loader failure	Specialized loader reports a format-specific issue or mismatch.
Recovery	Retry, fallback asset, or deferred failure reporting when policy allows.

Errors should preserve the asset identity, the loader identity, and the stage at which the failure occurred. That gives later milestones, tooling, and editor workflows enough information to explain the problem without guessing.

10. Integration with Engine Subsystems

Every major subsystem depends on assets, but assets do not depend on subsystem policy.

The asset system is a shared dependency for the framework, runtime, renderer, animation, physics, audio, gameplay, editor, networking, and save system. Each of those subsystems should consume assets through the asset service and handle asset handles or descriptors rather than reaching directly into storage.

The integration rule is asymmetrical. The asset system may expose resource lifecycle and metadata, but it must not take on the behavior of the consumer. Rendering should not be encoded into the asset system. Animation should not change the meaning of the asset cache. Physics should not redefine asset validation. Gameplay should not own loading policy.

Subsystem	Interaction rule
Framework	Provides service lifecycle, dependency injection, and asset system startup/shutdown.
Runtime	Uses handles and descriptors to consume loaded resources during execution.
Renderer	Consumes renderable assets but does not own their lifecycle rules.
Animation	Depends on model, texture, and material assets without changing asset ownership semantics.
Physics	May use model or material metadata, but not as a lifecycle owner.
Audio	Consumes audio assets and may request streaming readiness.
Gameplay	Uses asset handles for content, data, and references without directly loading files.
Editor	Uses registry, descriptors, and validation for browsing and import workflows.
Networking	May request remote or packaged assets later through the same contract model.
Save System	May serialize asset identifiers and handle references, not live payload ownership.

The most important integration rule is that assets are data and lifecycle, not rendering behavior. The asset system prepares resources; other systems decide how to use them.

11. Architecture Decisions and Tradeons

The design favors explicit ownership, predictable loading, and future packaging flexibility.

D003 deliberately chooses explicit descriptors, handles, and a centralized registry because those ideas prevent duplicate loads and hidden ownership. A common engine mistake is to let every subsystem load files independently; another is to make the asset layer a giant monolith that owns rendering or gameplay semantics. D003 avoids both mistakes.

Reference counting is chosen because many engine systems can share the same resource safely. Lazy loading is allowed because not every asset must be resident from the start. Preloading is allowed because some assets are needed early. The tradeoff is that the system must be more disciplined about lifecycle and validation.

Principle	Why it exists
SOLID	Each asset class and loader should own one clear responsibility.
Dependency Injection	Loaders, registry, cache, and service dependencies are supplied by composition.
Single Responsibility	AssetManager orchestrates; loaders load; cache caches; handles handle.
Composition	The framework composes asset services rather than letting them locate each other.
Loose coupling	Subsytems depend on handles and descriptors, not on file-system assumptions.
One class per file	Keeps the asset architecture easy to review and freeze.
Private fields	Internal state stays internal unless the contract explicitly exposes it.
ES Modules	Keeps dependency direction visible in the source tree.
No globals	Prevents hidden asset state from leaking across subsystems.

The key tradeoff is flexibility versus clarity. D003 chooses clarity. It prefers a small set of dependable contracts over a deeply clever but opaque loading system. That is the correct choice for a canonical architecture document.

12. Future Expansion

The architecture should be ready for bundles, remote assets, hot reload, and asset databases.

D003 should not assume the final shape of the asset ecosystem. It should, however, make space for future capabilities without redesigning the core asset contracts. Asset bundles, virtual assets, remote assets, cloud assets, hot reload, editor import tooling, asset database support, and build pipeline integration should all be able to attach through the same ownership model.

This future expansion model is one reason the descriptor and registry layers matter so much. If the metadata layer is strong enough, the system can evolve from local files to packaged bundles or remote resources without invalidating the meaning of an asset handle.

Future area	How it plugs in	Constraint
Asset bundles	Group assets for packaging and deployment.	Registry and metadata already support grouping.
Virtual assets	Represent generated or synthetic resources.	Descriptor model can identify non-file origins.
Remote assets	Load content from remote locations or services.	Handles and loaders can remain unchanged.
Cloud assets	Support network-backed distribution or streaming.	Caching and reference tracking remain valid.
Hot reload	Refresh assets during editor or development workflows.	Validation and registry refresh hooks are needed.
Editor integration	Browse, import, inspect, and validate assets.	Descriptors provide the authoritative view.
Asset database	Track asset provenance and dependency closure.	Registry is the conceptual seed of the database.
Build pipeline	Stage, transform, compress, and package assets.	Import metadata and platform variants make this possible.

The extension rule is simple: future systems may add new loaders, new package forms, or new delivery mechanisms, but they must not change the meaning of the asset lifecycle or break reference semantics.

13. Verification and Freeze Criteria

The architecture is ready when the team can reason about assets without implementation detail.

D003 should only be frozen when the team can explain the asset system through its contracts: what an asset is, how it is discovered, how it is imported, how it is cached, how it is referenced, and how it is released. If any of those answers require implementation code to explain, the architecture is not yet stable enough.

Review should focus on ownership, dependency direction, metadata, and lifecycle policy. Implementation-level optimizations belong to later work. The architecture document should instead define the non-negotiable behavior that later code must obey.

Architecture checklist

• AssetManager, AssetRegistry, AssetLoader, AssetCache, AssetService, AssetHandle, AssetDescriptor, AssetState, and AssetType each have a single documented ownership boundary

• loader architecture distinguishes generic loader policy from specialized format loaders

• asset lifecycle is explicit from discovery to cleanup

• metadata includes source, import, runtime, version, variants, and dependency information

• error handling covers invalid, missing, corrupted, circular, and loader failures

• integration rules keep the asset layer independent of rendering, ECS, physics, and gameplay

• future expansion does not require redefining handles or descriptors

Freeze checklist

• the dependency graph is documented and consistent

• memory ownership and reference counting rules are clear

• loading and caching policy are stable enough for downstream milestones

• the asset system can support editor, runtime, and build pipeline integration without rearchitecture

Review checklist

• reviewers can trace an asset from discovery to unload

• reviewers can explain why handles do not own asset memory

• reviewers can identify where new loaders are added

• reviewers can separate asset lifecycle from render-time usage

D003 is frozen when the asset system can be summarized as a centralized, dependency-injected, reference-aware lifecycle manager for all engine resources.

14. Canonical Summary

The asset system contract in one short statement for future implementers.

Echo Engine D003 defines a central asset system: AssetManager orchestrates, AssetRegistry catalogs, AssetLoader and specialized loaders import and validate, AssetCache prevents duplicates and supports reuse, AssetService exposes the system to the framework, AssetHandle gives consumers safe access, AssetDescriptor and AssetState describe identity and lifecycle, and AssetType routes loading policy. The system owns lifecycle, not rendering.

Future implementation AIs must treat this document as authoritative. If a later design conflicts with D003, the architecture specification wins unless it is formally revised.

Rule	Statement
Canonical rule	Prefer explicit asset contracts over hidden resource access.
Canonical ownership	Asset system owns lifecycle; consumers own usage.
Canonical boundary	Rendering and gameplay consume assets, but do not define them.
Canonical outcome	A maintainable and scalable asset backbone for the engine.

This closes the D003 architecture specification.